第二章 线性表 · 实践部分 教师版讲解

> 适用对象：授课教师（备课 / 课堂讲解 / 上机指导）
> 配套文档：《第二章 线性表的结构分析与应用-理论部分》《第二章 线性表的结构分析与应用-实践部分》《第二章 线性表的结构分析与应用-C语言知识准备》
> 讲解内容：案例一 学生成绩表（顺序表）、案例二 教师名单（单链表）
> 建议课时：**4 课时**（案例一 2 课时 + 案例二 2 课时），若课时紧张可按"2 课时压缩方案"执行（见第二节）

一、教学目标与重难点

1.1 本节要达成的目标

维度	目标	检验方式
知识	掌握顺序表、单链表的类型定义与存储特点	能默写 `SqList`、`LNode` 定义
知识	掌握两种结构下插入、删除、查找的算法步骤	能口述"几步、每步做什么"
能力	能独立写出插入 / 删除函数并调试通过	上机验收
能力	能根据场景选择顺序表或链表	课堂提问 / 作业
素养	严谨细致（边界条件差 1 就全错）；规范释放内存	代码检查

1.2 重点与难点

类型	内容	突破方法
重点	顺序表插入"从后往前搬"、删除"从前往后搬"	现场画格子演示，先手工搬一遍再写代码
重点	链表插入"先接后、再断前"、删除"摘链 + free"	用绳子/卡片模拟，固定成口诀
难点	位序 i ↔ 下标 i-1	全课反复强调，每次写代码前先问"第 i 个在哪"
难点	`InitList(LinkList *L)` 二级指针	与"传值改不动外面"实验呼应，不必深挖，先记住写法
难点	`malloc` / `free` 的时机与野指针	画"结点的一生"流程图

二、课时安排

方案 A：4 课时（推荐）

课时	内容	关键产出
第 1 课时	案例一：数据结构 + 初始化 + 插入算法	能画出插入的元素移动过程
第 2 课时	案例一：删除 + 查找 + 遍历 + 上机跑通	顺序表程序可运行
第 3 课时	案例二：结点定义 + 初始化 + 尾插建表	能画出链表结构图
第 4 课时	案例二：插入 + 删除 + 遍历 + 对比总结	链表程序可运行，能说出选型理由

方案 B：2 课时（压缩）

- 第 1 课时：只讲顺序表的**插入 + 删除**（跳过查找遍历细节，让学生自己读）；
- 第 2 课时：只讲链表的**插入 + 删除**，用"与顺序表对比"的方式快速带过；
- 建表与遍历留作上机任务，教师巡回指导。

三、课前准备

1. **环境**：Dev-C++ / Code::Blocks / VS Code + MinGW 均可；提醒学生**英文输入法**（中文逗号是本节最高频的坑）。
2. **提前准备好的输入样例**（写在 PPT 或板书角落，见第十节），避免课堂现场编数据浪费时间。
3. **道具/图示**：
   - 顺序表：画 5 个格子的"柜子"，用磁贴表示元素；
   - 链表：画 3 个带箭头的方框，强调"方框可以不挨着"。
4. **预习检查**（2 分钟）：提问结构体定义、`typedef`、数组下标从 0 开始、`malloc` 四件套。

四、案例一：学生成绩表（顺序表）讲解流程

4.1 导入（3 分钟）

> "一个班 40 人的成绩，用数组存够不够？为什么还要再包一层结构体？"

引出：**数组只能存数据，存不下"现在有几个"**——所以需要"仓库 + 计数器"。

4.2 数据结构定义（8 分钟）· 41-49 行

```
typedef struct {
    long id;                // 学号
    int  score;             // 成绩
} ElemType;                 // 数据元素：一名学生的成绩信息

typedef struct {
    ElemType data[MAXSIZE]; // 数组定义，用来顺序存储多个学生成绩
    int      length;        // 当前学生人数
} SqList;
```

讲解要点：

- `ElemType` = 数据元素（一个学生）；`SqList` = 表（花名册本子）；
- **必讲一句话**：`data[length]` 及以后是垃圾值，所以遍历只能是 `i < length`；
- 提问："`MAXSIZE=100`，现在 `length=3`，请问 `data[3]` 能读吗？"（答：不能，是空位）

4.3 初始化（3 分钟）· 52-54 行

- 只做 `length = 0`，**不用清数组**；
- 强调参数 `SqList *L` 与调用 `InitList(&L)` 的配对关系。

4.4 插入算法（15 分钟，本节第一个重点）· 57-71 行

讲解顺序建议（不要直接念代码）：

1. **先问**：要在第 2 个位置插一个人，得先干什么？（答：把第 2 个及以后的人往后挪一格）
2. **再画**：在黑板上画 `[1001][1002][1003]`，演示"从前搬"会覆盖数据，"从后搬"才对；
3. **后写代码**：对照图写出 `for (j = length-1; j >= i-1; j--) data[j+1] = data[j];`
4. **最后落三个判断**：位置合法？表满？搬完填空 + `length++`

板书模板：

```
插入位置 i=2，length=3
原表：   [A][B][C][ ]
              ↑ 空位在 length 处
从后往前搬：j 从 2（最后一个）开始
[A][B][C][C] → [A][B][B][C] → [A][ ][B][C]
填空：   [A][X][B][C]   length: 3 → 4
```

必强调的两个边界：

判断	条件	原因
位置合法	`1 <= i <= length+1`	可以插到表尾之后
表满	`length >= MAXSIZE`	柜子满了

4.5 删除算法（12 分钟）· 74-85 行

与插入对照着讲：

	插入	删除
判位置	`i <= length+1`	`i <= length`
搬的方向	从后往前（后移）	从前往后（前移）
循环	`j=length-1; j>=i-1; j--`	`j=i; j<length; j++`
赋值	`data[j+1] = data[j]`	`data[j-1] = data[j]`
收尾	`data[i-1]=e; length++`	`length--`

输出参数 `ElemType *e` 的讲法：

> "删除要告诉调用者两件事：**删成功没有**（用返回值），**删的是谁**（用 `&e` 带回来）。函数只能 `return` 一个值，所以第二个结果必须靠指针。"

对应调用：`ListDelete(&L, 3, &e)`。

4.6 查找与遍历（5 分钟）· 88-105 行

- `LocateElem` 返回**位序** `i+1`，找不到返回 `0`；
- 参数是 `SqList L`（不传址）：只读不改；
- 可顺带提问："班级 1000 人时，传值会怎样？"（答：整表拷贝，浪费；工程上应写 `const SqList *L`）

4.7 上机演示（12 分钟）· 107-151 行

按第十节的脚本现场跑一遍，重点让学生观察：

1. 建表循环里 `ListInsert(&L, L.length + 1, e)` —— **永远插在表尾**；
2. 插入 1009 到第 2 位后，**第 3 位变成了原来的 1002** —— 位数会变；
3. 删除第 3 位时删掉的是 1002（不是 1003）。

4.8 顺序表易错点清单（5 分钟）

#	错误写法	后果	正确写法
1	`for (i=0; i<=L.length; i++)`	多读一个垃圾值	`i < L.length`
2	插入判位置写 `i > L->length`	插不到表尾	`i > L->length + 1`
3	插入从前往后搬	覆盖数据	从后往前
4	`InitList(L)`	改不动外面	`InitList(&L)`
5	`L->data[i]` 当第 i 个	整体错位	`L->data[i-1]`
6	`L.length`（L 是指针时）	编译报错	`L->length`

五、案例二：教师名单（单链表）讲解流程

5.1 导入（3 分钟）

> "成绩表人数固定，用柜子没问题。教师名单随时有人调入调出，柜子还合适吗？"

引出：需要"来一个申请一块空间"，即链式存储。

5.2 结点定义（8 分钟）· 188-193 行

```
typedef struct LNode {
    long id;                // 工号
    char name[20];          // 姓名
    int  pms;               // 权限：1 = 教学督导，2 = 普通教师
    struct LNode *next;     // 指针域：指向下一个结点
} LNode, *LinkList;
```

两个名字对照表（必讲）：

写法	含义	声明变量
`LNode`	结构体类型	`LNode e;` → 一个结点
`*LinkList`	指向结点的指针类型	`LinkList L;` = `LNode *L;`

必问："`next` 为什么只能是指针？"（答：写成 `LNode next` 就是自己套自己，大小无限递归，编译报错）

5.3 初始化与头结点（10 分钟，难点）· 196-203 行

- 二级指针 `LinkList *L`：因为要让外面的 `L` 指向新申请的头结点；
- **若学生听不懂，就用这句话兜底**："凡是初始化，调用就写 `InitList(&L)`，记住就行"；
- 头结点的作用：统一插入/删除逻辑，不用单独讨论"在第一个位置操作"的特例；
- 提醒：`(*L)->next` 的括号不能省。

5.4 尾插法建表（12 分钟）· 206-217 行

讲法：先讲"笨办法"再优化。

- 笨办法：每来一个人，从头走到尾再接上 → 建 n 个人要 $O(n^2)$；
- 优化：用 `r` 记住表尾 → $O(n)$。

三步口诀： `r->next = s`（接上）→ `r = s`（尾指针跟过去）→ 最后 `r->next = NULL`（封口）

必强调：

- 最后那句 `r->next = NULL` **不能省**（新结点 `next` 是垃圾值，遍历会崩）；
- `scanf("%ld,%s,%d", &s->id, s->name, &s->pms)` 里 `s->name` **不加** `&`；
- 姓名里不能有空格（`%s` 遇空格截断）。

可选补充（1 分钟）：头插法 `s->next = L->next; L->next = s;` 会得到**逆序**的表，可与尾插法对比。

5.5 插入（15 分钟，本节第二个重点）· 237-253 行

两句核心（必须背，顺序不能反）：

```
s->next = p->next;      // ① 先接后：新结点抓住后面的链
p->next = s;            // ② 再断前：前驱改指向新结点
```

错误演示法（效果很好）：故意先写 `p->next = s;` 再写 `s->next = p->next;`，让学生说后果（答：`s->next` 指向自己，后半条链丢失）。

细节提醒：

- `strcpy(s->name, e.name)` —— 字符数组不能 `=` 赋值；
- 为什么比顺序表快：不搬数据，只改 2 个指针（代价是找前驱 O(n)）。

5.6 删除（12 分钟）· 256-267 行

四步：找前驱 → 备份 `*e = *q` → 摘链 `p->next = q->next` → `free(q)`

必强调：

- `*e = *q` 必须在 `free(q)` **之前**（free 之后再读是未定义行为）；
- 忘了 `free` = 内存泄漏（这是考试最常考的改错点）；
- `p == NULL || p->next == NULL` 两个判空缺一不可（前者位置非法，后者第 i 个不存在）。

可补充的拓展（1 分钟）：`*e = *q` 会把 `next` 也拷过来，但它指向的结点可能已被摘链，所以打印 `e` 时只用 `id` / `name` / `pms`，**不要用** `e.next`。

5.7 遍历与权限判断（5 分钟）· 270-278 行

- 起点 `L->next`（跳过头结点），条件 `p != NULL`；
- 提问："写成 `p->next != NULL` 会怎样？"（答：漏掉最后一个教师）
- `p->pms == 1 ? "教学督导" : "普通教师"` —— 三元运算符做"编码 → 文字"的翻译，可让学生改成 `if-else` 再对比。

5.8 链表易错点清单（5 分钟）

#	错误写法	后果	正确写法
1	先 `p->next=s` 再 `s->next=p->next`	后半条链丢失	**先接后，再断前**
2	删除后不 `free(q)`	内存泄漏	`free(q); q = NULL;`
3	`s->name = e.name`	编译报错	`strcpy`
4	`while (p->next != NULL)`	漏掉最后结点	`while (p != NULL)`
5	`InitList(L)`	外面 L 不变	`InitList(&L)`

#	错误写法	后果	正确写法
6	`malloc` 后不判空	内存不足时崩溃	`if (s == NULL) return 0;`

六、对比总结与选型（10 分钟，建议板书）

操作	顺序表	单链表
判满	`length >= MAXSIZE`	一般不判
判空	`length == 0`	`L->next == NULL`
找第 i 个	`data[i-1]`，**O(1) 随机访问**	`GetElem` 顺着走，**O(n)**
插入	从后往前搬，O(n)	改 2 个指针，O(1)（+找前驱）
删除	从前往后搬，O(n)	摘链 + free，O(1)（+找前驱）
空间	一次开好，可能浪费	用多少申请多少
适合	人数固定、查得多	人数不定、增删多

结论（要求学生能复述）： 成绩表 → 顺序表；教师名单 → 单链表。

七、课堂提问设计（含参考答案）

#	提问时机	问题	参考答案
1	讲插入前	要在第 2 位插一个人，第一步做什么？	把第 2 位及以后的元素往后挪
2	讲插入后	顺序表插入为什么必须从后往前搬？	从前搬会覆盖未搬的数据
3	讲删除时	插入位置上限是 `length+1`，删除为什么是 `length`？	删除只能删已存在的元素
4	讲查找时	`LocateElem` 参数为什么不写 `SqList *L`？	只读不改，传值即可
5	链表初始化	为什么 `InitList(&L)` 用二级指针，而插入只要一级？	初始化要改指针变量本身
6	尾插法	最后那句 `r->next = NULL` 去掉会怎样？	表尾 `next` 是垃圾值，遍历崩溃
7	链表插入	两句改指针能不能调换顺序？	不能，先改 `p->next` 会丢失后半条链
8	链表删除	`free(q)` 能不能放在 `*e = *q` 前面？	不能，free 后读数据是未定义行为
9	总结时	场景：图书借阅记录，每天频繁借还、查询较少，选哪种？	单链表（增删频繁）

八、学生上机常见错误与应对

现象	原因	教师处理
输入 `1001，92` 后读数异常 / 卡住	用了中文逗号	提示语给英文示例；强调切输入法
姓名"张 三"只显示"张"	`%s` 遇空格结束	姓名不带空格（或改用 `fgets`）
遍历最后多出乱码行 / 程序崩溃	表尾 `next` 未置 `NULL`，或 `i <= length` 越界	检查 `r->next = NULL` 与循环边界
`expected ')' before ';'` 一类报错	`L->` 写成 `L.`，或括号不匹配	强调"变量用 `.`，指针用 `->`"
编译通过但插入后数据没变	忘了 `length++` 或忘了传 `&`	单步调试观察 `length`
重复运行后结果异常	顺序表未重新初始化	强调 `InitList(&L)` 是每轮第一步

九、分层教学建议

学生层次	要求	支持措施
基础薄弱	只完成顺序表的初始化、插入、删除；链表能跑通建表和遍历	提供**填空版代码**（关键行留空），先补空再独立写
中等	两个案例全部函数独立写出，能解释每步	按要求完成，鼓励加"按姓名查找"
学有余力	增加：① 链表销毁 `DestroyList`；② 有序插入；③ 用两个案例做性能对比测试	布置拓展任务，作为平时分加分项

> **填空版代码建议留空处**：插入的循环三行、删除的循环两行、链表的两句改指针、`free(q)`。

十、课堂演示脚本（可直接复制使用）

案例一 顺序表

输入：

```
请输入学生人数: 3
请依次输入学号和成绩(如 1001,92):
1001,92
1002,78
1003,85
请输入要查找的学号: 1003
```

关键输出：

```
--- 初始成绩表 ---
学号      成绩
1001     92
1002     78
1003     85
--- 插入成功，当前成绩表 ---
1001     92
1009     88
1002     78
1003     85
已删除学生：1002，成绩：78
--- 删除后成绩表 ---
1001     92
1009     88
1003     85
找到该学生，位于第 3 个位置，成绩 85
```

讲评点： 插入后"第 3 位"从 1003 变成了 1002，所以删除第 3 位删掉的是 1002——**位数会随插入而变。**

案例二 单链表

输入：

```
请输入教师人数: 2
请依次输入工号、姓名、权限(如 2001,张三,1):
2001,张三,1
2002,王五,2
```

关键输出：

```
--- 当前教师名单 ---
工号      姓名      权限
2001     张三      教学督导
2002     王五      普通教师
--- 插入成功，当前教师名单 ---
2001     张三      教学督导
2008     李四      普通教师
2002     王五      普通教师
已删除教师：2001 张三
--- 删除后教师名单 ---
2008     李四      普通教师
2002     王五      普通教师
```

讲评点： `main` 结束时没有释放链表，可顺势提问"程序结束前还应该做什么？"（引出 `DestroyList`，作为学有余力学生的拓展任务）。

十一、板书设计

```
                第二章 实践：线性表的两种实现
┌──────────────────────────────┬──────────────────────────────┐
|  顺序表 SqList               |  单链表 LinkList             |
|  data[MAXSIZE] + length      |  LNode{数据; *next} + 头结点 |
├──────────────────────────────┼──────────────────────────────┤
|  第 i 个 → data[i-1]         |  第 i 个 → GetElem(L,i)      |
|  插入：从后往前搬            |  插入：先接后，再断前        |
|   for(j=len-1;j>=i-1;j--)    |   s->next=p->next; p->next=s|
|   data[j+1]=data[j]          |  删除：*e=*q → 摘链 → free   |
|  删除：从前往后搬            |  建表：尾插 r->next=s; r=s   |
|   for(j=i;j<len;j++)         |        末了 r->next=NULL     |
|   data[j-1]=data[j]          |                              |
├──────────────────────────────┼──────────────────────────────┤
|  O(n)：赢在随机访问          |  O(1)改指针：赢在增删        |
└──────────────────────────────┴──────────────────────────────┘
选型：人数固定、查得多 → 顺序表；人数不定、增删多 → 单链表
```

十二、作业与评价

作业：

1. 默写顺序表 `ListInsert`、`ListDelete` 与链表 `ListInsert`、`ListDelete`（**必做**）；
2. 为案例一增加"按成绩区间统计人数"，为案例二增加"按姓名查找"（**提高**）；
3. 编写 `DestroyList` 释放整条链表，并说明不释放会怎样（**选做**）。

评价：

维度	权重	依据
代码可运行（两个案例）	50%	上机验收
关键算法口述（搬数据方向 / 改指针顺序）	20%	课堂提问
边界处理（判空、判满、判位置）	20%	代码检查
拓展任务	10%	加分

十三、教学反思要点

1. **位序 / 下标** 是否还有学生混淆？可在下节课开头用 3 道小题快速复检。
2. 链表**改指针顺序**仍有错误时，可让学生用"卡片 + 绳子"实物模拟一次。
3. 二级指针若普遍听不懂，采用"先记写法、后补原理"策略，避免卡住整节课进度。
4. 上机环节建议**先给填空版代码**再给空白版，减少基础薄弱学生的挫败感。
5. 记录学生高频报错（中文逗号、`i <= length`、`free` 遗漏），下次课前 2 分钟集中提醒。